

Seguridad en Spring Boot

Implementación de autenticación y protección en aplicaciones e-commerce



Contexto del Proyecto E-Commerce

Estamos trabajando con una aplicación completa de comercio electrónico que requiere integrar capas de seguridad robustas para proteger tanto a usuarios como a la plataforma.



Frontend React

Interfaz de usuario moderna con catálogo de productos, carrito de compras y gestión de pedidos



Backend Spring Boot

API REST que proporciona endpoints para CRUD de productos, gestión de usuarios y procesamiento de pedidos



Base de Datos MySQL

Persistencia con Spring Data JPA: entidades Producto, Categoría, Usuario y Pedido

¿Qué es Spring Security?



Spring Security es el framework de seguridad estándar para aplicaciones Spring. Proporciona una arquitectura completa y extensible para proteger aplicaciones empresariales contra las amenazas más comunes.

Actúa como una capa de defensa integral que intercepta las peticiones HTTP antes de que lleguen a tus controladores, verificando identidades y permisos de forma transparente.

Tres Pilares de Spring Security



Autenticación

¿Quién eres? Verifica la identidad del usuario mediante credenciales (usuario/contraseña, tokens, certificados). Asegura que quien accede al sistema es realmente quien dice ser.



Autorización

¿Qué puedes hacer? Una vez autenticado, determina qué recursos y operaciones puede acceder el usuario según sus roles y permisos asignados.



Protección contra Ataques

Defensa integrada contra vulnerabilidades comunes: Cross-Site Scripting (XSS), CSRF, inyección SQL, ataques de fuerza bruta y otras amenazas de seguridad.

Aplicación en Nuestro E-Commerce

Protección de Recursos

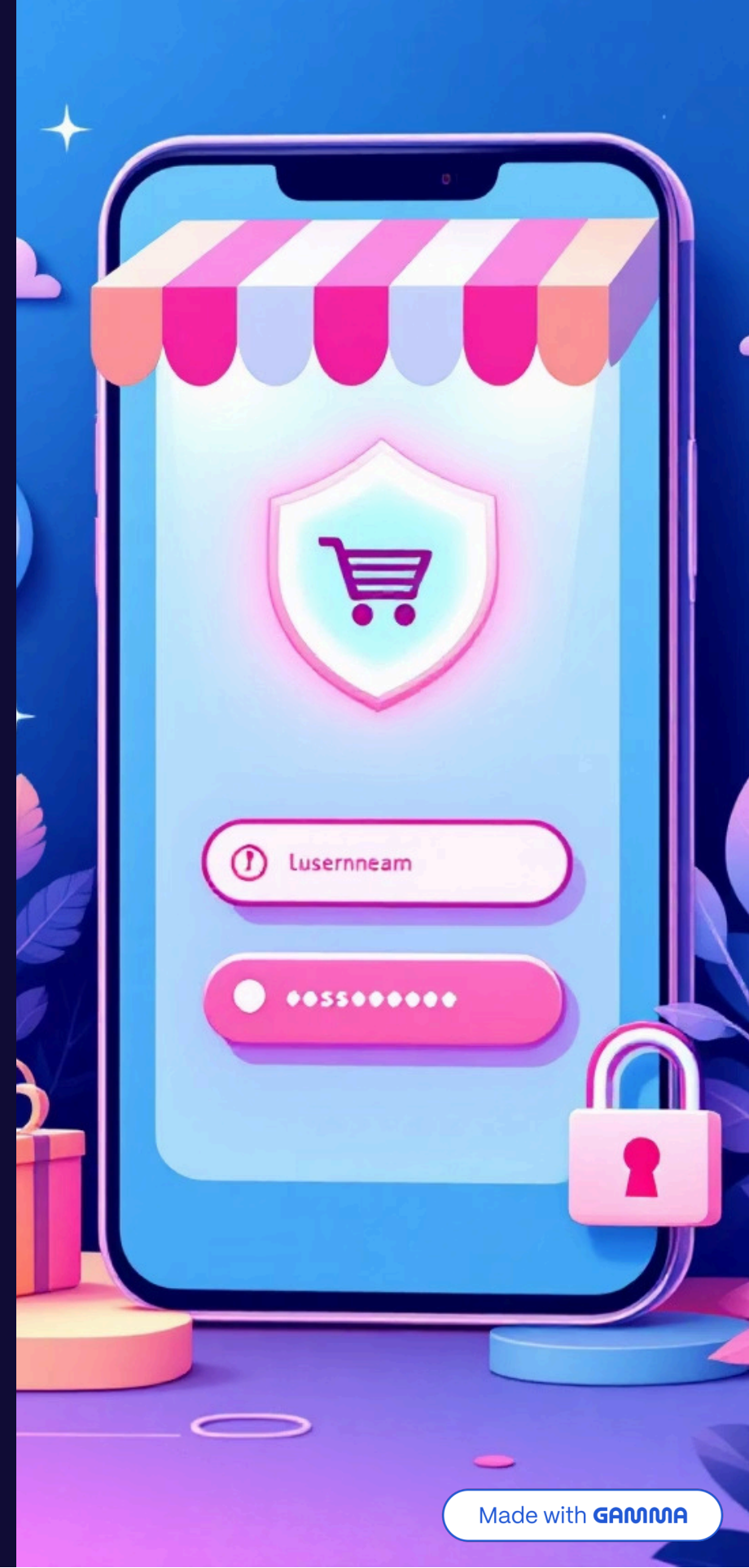
- Evitar que usuarios no autenticados realicen pedidos o accedan a información sensible
- Restringir operaciones críticas: solo administradores pueden eliminar productos o modificar el catálogo
- Proteger endpoints de gestión de usuarios y configuración del sistema

Gestión de Usuarios

- Sistema de login/logout seguro con manejo de sesiones
- Asignación de roles diferenciados (USER para clientes, ADMIN para administradores)
- Recuperación segura de contraseñas y gestión de credenciales

Defensa de la Aplicación

- Protección automática contra XSS que podría robar cookies de sesión
- Prevención de CSRF en formularios y operaciones sensibles
- Límites de intentos de login para evitar ataques de fuerza bruta



Pasos para Implementar Spring Security

01

Añadir Dependencias Maven

Incluir spring-boot-starter-security en pom.xml para importar todas las bibliotecas necesarias del framework

02

Crear Clase de Configuración

Desarrollar SecurityConfig.java anotada con @Configuration y @EnableWebSecurity para definir las reglas de seguridad

03

Definir Usuarios y Roles

Configurar el sistema de usuarios, ya sea en memoria o conectado a base de datos, asignando roles apropiados

04

Configurar Autenticación/Autorización

Spring Security maneja automáticamente el flujo de login, creación de sesiones, validación de credenciales y aplicación de reglas

SecurityConfig.java: El Corazón de la Configuración

¿Qué controla?

- **Endpoints públicos:** Acceso sin autenticación (catálogo, landing page)
- **Endpoints protegidos:** Requieren login activo
- **Endpoints con roles:** Solo ADMIN puede acceder
- **Mecanismos de defensa:** Configuración de protecciones contra ataques

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/productos/**")
                .permitAll()
                .requestMatchers("/api/admin/**")
                .hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

Esta clase define las reglas de seguridad fundamentales para tu API. Cada endpoint puede tener diferentes niveles de protección según las necesidades del negocio.

Protección contra CSRF en APIs

¿Qué es CSRF?

Cross-Site Request Forgery (Falsificación de Petición en Sitios Cruzados) es un ataque donde un sitio malicioso engaña al navegador del usuario para que envíe peticiones no autorizadas a tu aplicación.



1

Ataques Tradicionales

Se basan en cookies de sesión que el navegador envía automáticamente con cada petición, incluso desde sitios externos

2

APIs Stateless

Usan tokens (JWT) que deben enviarse explícitamente en headers, no se envían automáticamente

3

Protección Natural

En APIs basadas en tokens, CSRF se desactiva porque el vector de ataque principal (cookies automáticas) no existe

Autenticación vs Autorización

Autenticación

¿Eres quien dices ser?

Proceso de verificar la identidad del usuario mediante credenciales válidas

- Login con usuario/contraseña
- Validación de tokens JWT
- Verificación biométrica
- Autenticación multifactor

Autorización

¿Qué puedes hacer?

Determinar qué recursos y acciones están permitidos para un usuario autenticado

- Roles: USER, ADMIN, MANAGER
- Permisos granulares por endpoint
- Control de acceso a datos
- Restricciones por funcionalidad

📌 **Importante:** La autenticación siempre ocurre antes que la autorización. Primero verificamos quién eres, luego qué puedes hacer.

Resumen: Seguridad Integral en Spring Boot

Spring Security protege tu e-commerce en tres niveles

Autenticación de identidad, autorización de accesos y defensa contra vulnerabilidades comunes

La configuración es flexible y declarativa

SecurityConfig.java permite definir reglas específicas para cada endpoint según las necesidades del negocio

Las APIs stateless simplifican la seguridad

El uso de tokens JWT elimina vulnerabilidades relacionadas con sesiones y cookies, deshabilitando naturalmente CSRF

Con Spring Security correctamente configurado, tu aplicación e-commerce estará protegida contra las amenazas más comunes, permitiendo que usuarios y administradores operen de forma segura.